UNIVERSITY COLLEGE OF APPLIED SCIENCES / COMPUTER ENGINEERING DEPARTMENT

Robotics Programming

Course syllabus

Course code	DSAI 4304	**Term**	First Semester, 2026-2007
Credits	3 Hours	**Instructor**	Dr. Shadia Baroud
Meeting time	One weekly online session	**LMS/contact**	https://moodle.ucas.edu.ps/course/view.php?id=894
Delivery	Fully online	**Duration**	14 weeks

Course description

This course combines robotics theory with programming mobile robots using ROS 2. Students study the ROS graph and software workspace, then connect these ideas to robot motion, coordinate transformations, sensor measurements, feedback, path planning, mapping, localization and autonomous navigation. Python and C++ nodes, topics, services, actions, custom messages, parameters, and launch files provide the implementation foundation. Each week pairs an explicit concepts lesson with a simulation-first activity. The supplied slide sequence uses a differential-drive mobile robot as its main integration example. Physical hardware is not required for participation.

Learning outcomes

By the end of the course, students should be able to:

- Explain the role of ROS 2 nodes, topics, services, actions, packages and the ROS graph in a robot software system.
- Create and build ROS 2 workspaces and packages and implement runnable nodes in Python and C++.
- Design and test publisher/subscriber, service and action interactions, including a custom message interface.
- Configure nodes with parameters and launch files and organize a repeatable robot bring-up workflow.
- Parse and validate serial or sensor data and publish it using appropriate ROS 2 message types.
- Explain mobile-robot kinematics and coordinate transformations; describe a robot with URDF/Xacro and manage frames with TF2.
- Analyze sensor measurements and explain sampling, noise, calibration and basic sensor-fusion concepts.
- Compare Dijkstra/A* path planning, mapping, localization and SLAM and explain how these functions fit into a Nav2 workflow.
- Apply a simulation-based mapping, localization and navigation workflow using SLAM Toolbox, AMCL and Nav2.
- Debug, test and document a ROS 2 system so another person can reproduce and review it online.

Background and course materials

- **Recommended background.** Basic programming in Python or C++, and familiarity with files, folders, and command-line commands. Prior robotics or ROS experience is not required.
- **Primary text.** Thompson Carter, Rafael Sanders and Miguel Farmer, Advanced Robotics Programming with ROS2 and Python: Mastering Navigation, Sensing and Multi-Robot Systems in Real-World Applications (2025). Assigned chapters are listed in the weekly plan.
- **Course materials.** Instructor slide pack (Weeks 1–14), guided lab instructions, code examples and simulation or recorded sensor data distributed through the course LMS.

Online learning format

- **Weekly rhythm:** short preparation and a readiness check; a live online lesson with screen-shared coding or simulation; then a guided asynchronous lab and LMS submission.
- Live sessions focus on explanation, worked examples, debugging and questions. Lab tasks are designed to be completed individually with instructor support through the LMS or scheduled online help sessions.
- All core activities are simulation-first. Recorded demonstrations, ROS bag files or an instructor-hosted environment can provide a fallback when a student's computer or connection cannot run the full simulator.
- The LMS is the source for meeting links, announcements, due dates, submission instructions and any updated setup notes. Exact class times and office hours are to be confirmed by the instructor.

Software and setup baseline

The supplied slides target Ubuntu 24.04 and ROS 2 Jazzy. Use Gazebo Harmonic for the ROS 2 simulation exercises. Install instructions and package names can differ across ROS releases; where a book example targets ROS 2 Foxy or Gazebo Classic, follow the course setup guide and the assigned slide version instead.

Core environment	Ubuntu 24.04; ROS 2 Jazzy; Gazebo Harmonic; RViz2; colcon; Python 3; C++ toolchain; Git; and a code editor.
Online fallback	Use supplied ROS bag/serial data, streamed demonstrations, or an instructor-provided remote environment if local installation is not practical.

Assessment plan

The following weights form a proposed assessment plan and should be aligned with the approved UCAS course requirements before the syllabus is issued to students.

Component	Weight	Evidence
Weekly readiness checks	**10%**	Short online quizzes or check-ins on the week's concepts and setup.
ROS 2 programming lab portfolio	**35%**	Code, build/run evidence, brief explanation and reproducible instructions submitted through the LMS or course repository.
Midterm examination	**20%**	Individual online examination covering the first half of the course, through Week 7.
Final examination	**35%**	Cumulative individual online examination covering the course, with emphasis on integration, sensing and navigation.

Examination plan

- **Midterm examination.** Covers Weeks 1–7, including ROS 2 architecture and communication patterns, package/build structure, callback timing, control and serial-data concepts, parameters, launch files and the associated programming work.
- **Final examination.** Cumulative across the semester, with emphasis on Weeks 8–14: networked and embedded systems, kinematics and coordinate transforms, sensor measurement and fusion, graph-search planning, mapping, localization, SLAM and how these components fit into Nav2.
- **Online arrangements.** Exam dates, duration, question format, permitted resources and submission instructions will be announced in the LMS and follow UCAS assessment requirements.

Weekly course plan

Each week identifies a theory focus alongside its online activity. Conceptual understanding is reinforced through simulation and assessed in readiness checks and both examinations; lab submissions emphasize implementation and

explanation. Reading refers to the primary text unless marked as official documentation. Lab data, starter code and submission instructions will be posted in the LMS.

Week	Topics, online activity and preparation
Week 1	Course learning outcomes, schedule, assessment and online learning expectations; overview of robotic systems and the role of software; introduction to ROS 2 and the course simulation environment; check students' development setup.
Week 2	**ROS 2 foundations and environment setup** **Theory:** ROS 2 as robotics middleware; the robot sense–plan–act loop; nodes, interfaces, and the ROS graph. **Online activity:** Install and verify the course environment; run a demo node and inspect active nodes/topics. **Preparation:** Read Ch. 1–2 for concepts; use the course guide for the Jazzy setup.
Week 3	**Workspaces, packages, and Python nodes** **Theory:** Workspaces and packages as modular units; dependencies, package metadata, and the build/run lifecycle. **Online activity:** Use the ROS 2 CLI, create a colcon workspace and Python package, and build/run a minimal node. Submit the package structure and a short run log. **Preparation:** Read Ch. 2–3.
Week 4	**C++ nodes and timers** **Theory:** Event-driven execution; callbacks, timer periods, blocking versus non-blocking work, and Python/C++ implementation trade-offs. **Online activity:** Create an rclcpp package with a timer callback; publish periodic status data and inspect its rate from the CLI. **Preparation:** Read Ch. 3 and the assigned ROS 2 C++ tutorial.
Week 5	**Topics, publishers/subscribers, and custom messages** **Theory:** Publish/subscribe communication, typed message contracts, queueing, and ROS 2 quality-of-service choices. **Online activity:** Connect Python and C++ nodes using topics. Implement and test a small publisher/subscriber system with a custom interface. **Preparation:** Read Ch. 2–3 and the assigned ROS 2 QoS notes.
Week 6	**Serial communication and robot control integration** **Theory:** Open-loop versus closed-loop control; command and feedback paths; packet framing, timing and checksums as error detection. **Online activity:** Parse the supplied 17-byte sample packet or simulator stream and publish validated data. No Arduino hardware is required. **Preparation:** Read Ch. 4 movement concepts, selected Ch. 11 sections, and the Week 5 slides.
Week 7	**Services and actions** **Theory:** Choosing request/response services or long-running actions; action goals, feedback, cancellation and results. **Online activity:** Create a service and a simple action server/client; relate action feedback to a navigation goal. **Preparation:** Read Ch. 2 and the relevant Ch. 4 concepts.
Week 8	**Parameters, launch files and repeatable bring-up** **Theory:** Separating configuration from code; namespaces and remapping; composing a repeatable multi-node system. **Online activity:** Launch a small system with editable parameters. Midterm examination covers Weeks 1–7. **Preparation:** Read Ch. 2 and the Week 7 slides.

Week	Topics, online activity and preparation
Week 9	**Embedded and networked robot systems** **Theory:** Distributed robot architecture; ROS network discovery, latency and reliability; embedded resource constraints and multi-robot namespaces. **Online activity:** Inspect nodes across processes or analyze the instructor demonstration; complete a formative integration check. No physical board is required. **Preparation:** Read selected Ch. 7 and Ch. 11.
Week 10	**LiDAR, camera, and range sensor data** **Theory:** Sensor modalities and measurement properties: sampling rate, range, field of view, noise, units and frame IDs. **Online activity:** Explore a supplied topic or bag, identify message fields and rates, and document data quality. **Preparation:** Read Ch. 6 and the Week 9 slides.
Week 11	**Robot description, TF2, RViz2, and Gazebo** **Theory:** Differential-drive kinematics; 2D coordinate transformations; map, odom, and base frames; links, joints, and the TF tree. **Online activity:** Build or adapt a differential-drive URDF/Xacro model; inspect frames in TF2 and visualize it in RViz2 and Gazebo Harmonic. **Preparation:** Read Ch. 4.
Week 12	**Sensor integration and validation** **Theory:** Sensor calibration and uncertainty; time and frame alignment; the purpose of complementary and Kalman filtering in sensor fusion. **Online activity:** Replay or publish a sensor stream, verify units, timestamps, and frame IDs and compare raw and filtered data where supplied. **Preparation:** Read Ch. 6 and the Week 11 slides.
Week 13	**Path planning and embedded deployment** **Theory:** Graph-search planning; Dijkstra versus A* and the role of a heuristic; deployment constraints on networked embedded robots. **Online activity:** Trace a path through a small grid, then relate the result to a navigation costmap. Review a deployment checklist using a recorded or simulated setup. **Preparation:** Read Ch. 5 and selected Ch. 11.
Week 14	**Mapping, localization, Nav2, and final examination** **Theory:** Occupancy-grid mapping; localization versus SLAM; AMCL particle-filter intuition; roles of mapping, planners, and recovery behaviors in Nav2. **Online activity:** Run or review a simulation pipeline with SLAM Toolbox, AMCL, and Nav2; send and monitor a navigation goal. Complete course synthesis and the final examination. **Preparation:** Read Ch. 5–6 and the Nav2 documentation; Ch. 7–10 and Ch. 12 are optional enrichment.

Online participation and course policies

- **Participation.** Students should check the LMS regularly, contribute to live or asynchronous discussions, and complete the weekly practice. When connectivity or scheduling prevents live attendance, students should use the posted asynchronous alternative and contact the instructor through the LMS.
- **Collaboration and academic integrity.** Follow UCAS academic integrity requirements. Cite or acknowledge external code, datasets, and assets as required. Students must be able to explain and reproduce the work they submit; collaboration boundaries and any course-specific AI-use rules will be stated in the LMS.
- **Deadlines and accommodations.** Official UCAS policies govern late work, assessment accommodations, and appeals. Due dates and submission formats will be published in the LMS.
- **Safety and hardware.** No student is required to own, power or operate physical robot hardware. Any hardware-related content is taught through simulation, supplied data or an instructor demonstration. Follow instructor directions and manufacturer guidance during any optional hardware demonstration.

Exam coverage guide

Examinations assess conceptual understanding as well as the ability to interpret ROS 2 systems and their outputs.

Examination	Coverage	Review focus
Midterm	Weeks 1–7	ROS 2 architecture and graph; packages and build lifecycle; callbacks and timing; topic/message semantics; service versus action; open/closed-loop control; serial framing; parameters and launch.
Final	Cumulative, with emphasis on Weeks 8–14	Differential-drive kinematics and coordinate transforms; sensor properties, calibration, and fusion; Dijkstra/A*; mapping versus localization and SLAM; AMCL; network/embedded constraints; Nav2 roles.

References and technical resources

- Carter, T., Sanders, R., & Farmer, M. (2025). ***Advanced Robotics Programming with ROS2 and Python: Mastering Navigation, Sensing, and Multi-Robot Systems in Real-World Applications***. Primary text supplied for the course.
- Herath, D., & St-Onge, D. (Eds.). (2022). ***Foundations of robotics: A multidisciplinary approach with Python and ROS***. Springer. https://doi.org/10.1007/978-981-19-1983-1
- Lynch, K. M., & Park, F. C. (2017). ***Modern robotics: Mechanics, planning, and control***. Cambridge University Press. ISBN 978-1-107-15630-2.
- ***ROS 2 Jazzy documentation and Ubuntu installation guide***: https://docs.ros.org/en/jazzy/ and https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html
- ***Gazebo Harmonic documentation***, including ROS 2 integration: https://gazebosim.org/docs/harmonic/ and https://gazebosim.org/docs/harmonic/ros2_integration/
- **Nav2 documentation**: https://docs.nav2.org/
- **ROS 2 Jazzy SLAM Toolbox package documentation**: https://docs.ros.org/en/jazzy/p/slam_toolbox/